Experiment 1: Stateful vs Stateless Tool-use Evaluation

Purpose. This experiment asks whether final-state correctness is enough to evaluate a tool-use agent. The controlled comparison keeps the final goal comparable while changing whether the environment preserves intermediate state transitions, dependency failures, and recovery opportunities.

Dataset. The experiment uses two tracks. The synthetic track contains 9 hand-designed cases covering file indexing, stale search snapshots, issue workflow constraints, and calendar conflict recovery. The ToolSandbox track contains 40 comparison cases sampled from five task domains, with stateful trajectories preserving helper calls and stateless trajectories compressed into final-state actions.

Metrics table reading. The table reports call-level success, invalid call rate, recovery rate, trajectory length, final-state correctness, and trajectory divergence. The key pattern is that final-state correctness is identical, but the process-level measurements differ sharply.

Dataset	Setting	Cases	Success Rate	Invalid Calls	Recovery	Avg. Steps	Final State	Trajectory Divergence
Synthetic	Stateful	9	88.2%	N/A	100.0%	3.78	100.0%	N/A
Synthetic	Stateless	9	100.0%	N/A	0.0%	2.44	100.0%	N/A
ToolSandbox	Stateful	40	96.8%	0.0%	100.0%	3.15	100.0%	38 / 40
ToolSandbox	Stateless	40	100.0%	0.0%	0.0%	1.50	100.0%	38 / 40

Result interpretation. Both stateful and stateless settings reach 100% final-state correctness on the synthetic and ToolSandbox tracks. However, this equality is misleading: stateful execution exposes intermediate failures and recovery behavior, while the stateless baseline eliminates many opportunities for a failure to occur. In the ToolSandbox subset, 38 of 40 cases diverge at the trajectory level, showing that identical final states do not imply equivalent tool-use competence.

Conclusion. Final-state correctness alone is insufficient. A useful tool-use benchmark must also measure trajectory-level semantics such as dependency resolution, intermediate failure exposure, recovery, and state-sensitive workflow constraints.

Reporting note: synthetic cases isolate controlled mechanisms; ToolSandbox subsets provide a more realistic benchmark-distribution check. The two tracks should be read as complementary evidence.

Experiment 2: Cross-tool Dependency Difficulty Curve

Purpose. This experiment measures how different reasoning strategies degrade as tool-use tasks become more dependent on intermediate tool outputs and hidden side effects. Instead of reporting a single aggregate success rate, it constructs a difficulty curve from L1 single-tool tasks to L3 implicit side-effect dependency tasks.

Dataset. The synthetic benchmark contains 15 cases balanced across L1, L2, and L3. The ToolSandbox dependency subset contains 24 cases, with 8 cases per difficulty level. L1 contains single-tool tasks, L2 contains explicit helper-to-action chains, and L3 contains multi-step or implicit state dependencies such as recency, time, location, stale snapshots, and side-effect ordering.

Metrics table reading. The table reports per-strategy success at each difficulty level, the L1-to-L3 degradation, and the area under the difficulty curve. This makes the degradation pattern visible rather than hiding it inside an average.

Dataset	Strategy	L1 Success	L2 Success	L3 Success	L1-to-L3 Drop	AUC
Synthetic	direct	100.0%	20.0%	0.0%	100.0%	0.400
Synthetic	cot	100.0%	60.0%	40.0%	60.0%	0.667
Synthetic	react	100.0%	100.0%	80.0%	20.0%	0.933
Synthetic	self_refine	100.0%	100.0%	100.0%	0.0%	1.000
ToolSandbox	direct	100.0%	0.0%	0.0%	100.0%	0.333
ToolSandbox	cot	100.0%	100.0%	12.5%	87.5%	0.708
ToolSandbox	react	100.0%	100.0%	50.0%	50.0%	0.833
ToolSandbox	self_refine	100.0%	100.0%	100.0%	0.0%	1.000

Result interpretation. The same qualitative ordering appears in both datasets. Direct succeeds on L1 but collapses once prerequisites matter. CoT handles many explicit dependencies but degrades sharply on implicit side effects. ReAct is more stable because it preserves an observe-act loop. Self-Refine is strongest in these controlled policies, but its robustness comes with longer trajectories and more tool interactions.

Conclusion. Cross-tool dependency difficulty curves are more informative than aggregate success. They reveal not only which strategy wins overall, but where each strategy begins to fail as statefulness and dependency complexity increase.

Reporting note: synthetic cases isolate controlled mechanisms; ToolSandbox subsets provide a more realistic benchmark-distribution check. The two tracks should be read as complementary evidence.

Experiment 3: Noise Robustness

Purpose. This experiment evaluates whether stateful tool-use agents remain reliable when the environment is noisy. It separates execution-level failures, such as timeout and schema drift, from observation-level corruption, such as stale state, vague observations, and misleading observations.

Dataset. The fixed-trajectory layer reuses the Experiment 1 setup and produces 136 noise cases. The closed-loop policy layer contains 115 cases: 75 synthetic policy cases across task templates and noise intensities, plus 40 domain-balanced ToolSandbox policy cases. The closed-loop layer is the main robustness test because noisy observations can affect the next decision.

Metrics table reading. The table reports success@1, pass^k, recovery rate, cost increase, state corruption rate, and noise-specific pass^k. The contrast between fixed trajectories and closed-loop policies is the main result.

Layer / Strategy	Runs	Success@1	Pass^k	Recovery	Cost Increase	State Corruption	Timeout Pass^k	Stale Pass^k	Vague Pass^k	Misleading Pass^k
Fixed trajectory	136	52.9%	100.0%	100.0%	0.50	0.0%	100.0%	100.0%	100.0%	N/A
Policy: direct	115	7.8%	7.8%	0.0%	0.00	92.2%	0.0%	30.4%	4.3%	4.3%
Policy: cot	115	7.8%	73.0%	66.9%	0.53	27.0%	65.2%	30.4%	91.3%	100.0%
Policy: react	115	7.8%	76.5%	75.6%	1.20	23.5%	65.2%	95.7%	73.9%	69.6%
Policy: self_refine	115	7.8%	86.1%	85.6%	0.66	13.9%	65.2%	95.7%	91.3%	100.0%

Result interpretation. Fixed trajectories recover all cases with pass^k of 100% and zero state corruption, which confirms that simple retry can handle many injected execution faults when the future trajectory is fixed. The policy layer is much more revealing: direct is fragile across noise types, CoT recovers vague and misleading observations but is weak under stale state, ReAct is strongest on stale state through re-querying, and Self-Refine has the best average robustness across noise types.

Conclusion. Noise robustness is not just a retry problem. Stateful agents need observation validation, freshness checks, re-query behavior, and self-correction because corrupted observations can lead to wrong future actions even when the underlying state is not permanently corrupted.

Reporting note: synthetic cases isolate controlled mechanisms; ToolSandbox subsets provide a more realistic benchmark-distribution check. The two tracks should be read as complementary evidence.

Experiment 4: Backend Migration

Purpose. This experiment evaluates whether a strategy that succeeds in a simple mock backend still succeeds when the same task is executed under more realistic backend conditions. It treats backend realism as a separate axis from task semantics and noise injection.

Dataset. The benchmark contains 17 backend migration cases: 5 synthetic cases and 12 ToolSandbox-derived cases. The same cases are executed on three backends: mock, sandbox, and live-like. Sandbox now includes artifact-backed file/search behavior, while live-like adds deterministic timeout and schema-drift fault classes.

Metrics table reading. The table reports final correctness on each backend, the mock-to-live migration gap, live-like state divergence, live-like recovery, trajectory length, and live-like latency. The central measurement is whether mock success transfers to a backend with more realistic execution behavior.

Strategy	Mock Final	Sandbox Final	Live-like Final	Mock-to-Live Gap	Live Divergence	Live Recovery	Avg. Steps	Live Latency (ms)
direct	100.0%	100.0%	0.0%	100.0%	100.0%	0.0%	1.82	5.54
cot	100.0%	100.0%	47.1%	52.9%	52.9%	47.1%	2.82	11.11
react	100.0%	100.0%	76.5%	23.5%	23.5%	76.5%	3.82	16.84
self_refine	100.0%	100.0%	76.5%	23.5%	23.5%	76.5%	3.82	16.60

Result interpretation. All strategies remain correct under mock and sandbox, but live-like execution exposes large migration gaps. Direct falls to 0% because it has no recovery path. CoT recovers some transient failures but fails under more persistent faults. ReAct and Self-Refine transfer better because they have larger recovery budgets, but even they fail under hard schema drift in the current setup.

Conclusion. Mock-only evaluation can overestimate deployment robustness. The experiment shows why backend realism should be evaluated explicitly: correctness in a clean in-memory environment does not guarantee robustness under latency, transient failure, schema mismatch, and live-like execution faults.

Reporting note: synthetic cases isolate controlled mechanisms; ToolSandbox subsets provide a more realistic benchmark-distribution check. The two tracks should be read as complementary evidence.